



**Universidad Tecnológica Nacional
Facultad Regional Córdoba**

Algoritmos y Estructuras de Datos

Ficha Nro. 13

Arreglos: Algoritmos y Técnicas Básicas

Prof. Ing. Corso Cynthia

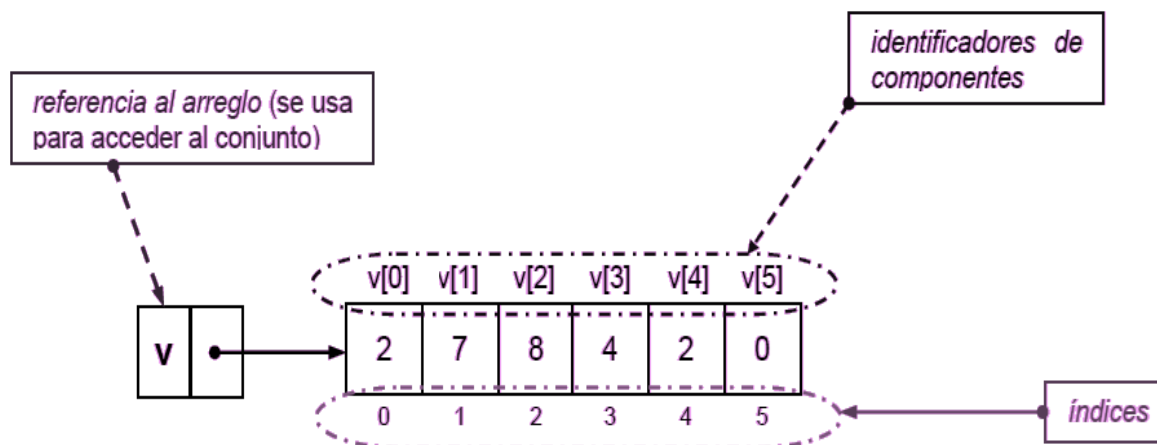
Temario

- Introducción. Revisión Ficha 12.
- Ordenamiento
- Búsquedas en Arreglos:
 - Secuencial
 - Búsqueda Binaria

Introducción

- Arreglo unidimensional: es una colección de valores que se organiza de tal forma, que cada componente individual es identificado por un número denominado **índice**.
- Python provee un tipo de secuencia que permite representar a un arreglo unidimensional, denominada **list** o **lista**. Se caracteriza por ser **mutable**.

Figura 1: Esquema básico de un arreglo unidimensional.



Arreglos unidimensionales: Creación

- La **creación del arreglo unidimensional** puede realizarse de diferentes maneras:

i) Creando un arreglo vacío. Por ejemplo:

```
notas = [] o notas = list()
```

ii) En el caso de conocer previamente los valores a almacenar, se puede crearlo **enumerando sus componentes**:

```
notas = [9, 10, 7, 6, 7]
```

iii) Estableciendo la cantidad de componentes del arreglo, y que almacene el mismo valor en cada casilla. Para esto se usa el operador `*`.

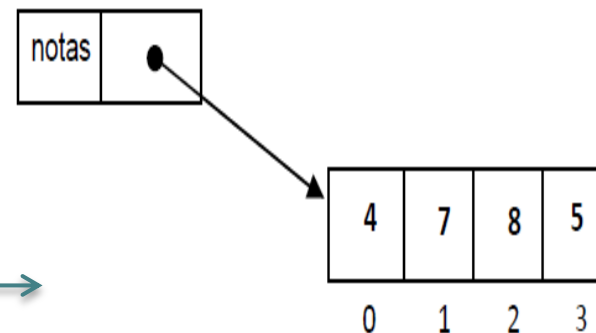
```
notas = 4 * [0]
```

Arreglos: Carga de elementos (I)

- Si el arreglo o lista se crea vacío, la carga se puede realizar por medio del método `append()`. Ejemplo:

```
notas = []  
notas.append(4)  
notas.append(7)  
notas.append(8)  
notas.append(5)  
print('Notas:', notas)
```

Figura 3: Efecto de la función `append()` al crear una variable de tipo `list`.



La función `append()` agrega el contenido de la variable que es tomado como parámetro a una lista, pero lo hace de forma que ese valor se agrega al final del arreglo o lista.

Arreglos: Carga de elementos (II)

- En el caso que el arreglo no se cree vacío, se puede agregar elementos solicitando al usuario que **cargue los datos por teclado**.
- En este caso para procesar un arreglo es común el uso del ciclo **for**.
- El siguiente script muestra la forma de hacer la carga en esta situación.

```
notas1 = 4 * [0] # Arreglo con 4 componentes.  
for i in range(4):  
    # Carga las notas por teclado  
    notas1[i]= int(input('Cargue nota: '))  
  
print('La carga del arreglo ha finalizado!')
```

Arreglos: Visualización de elementos

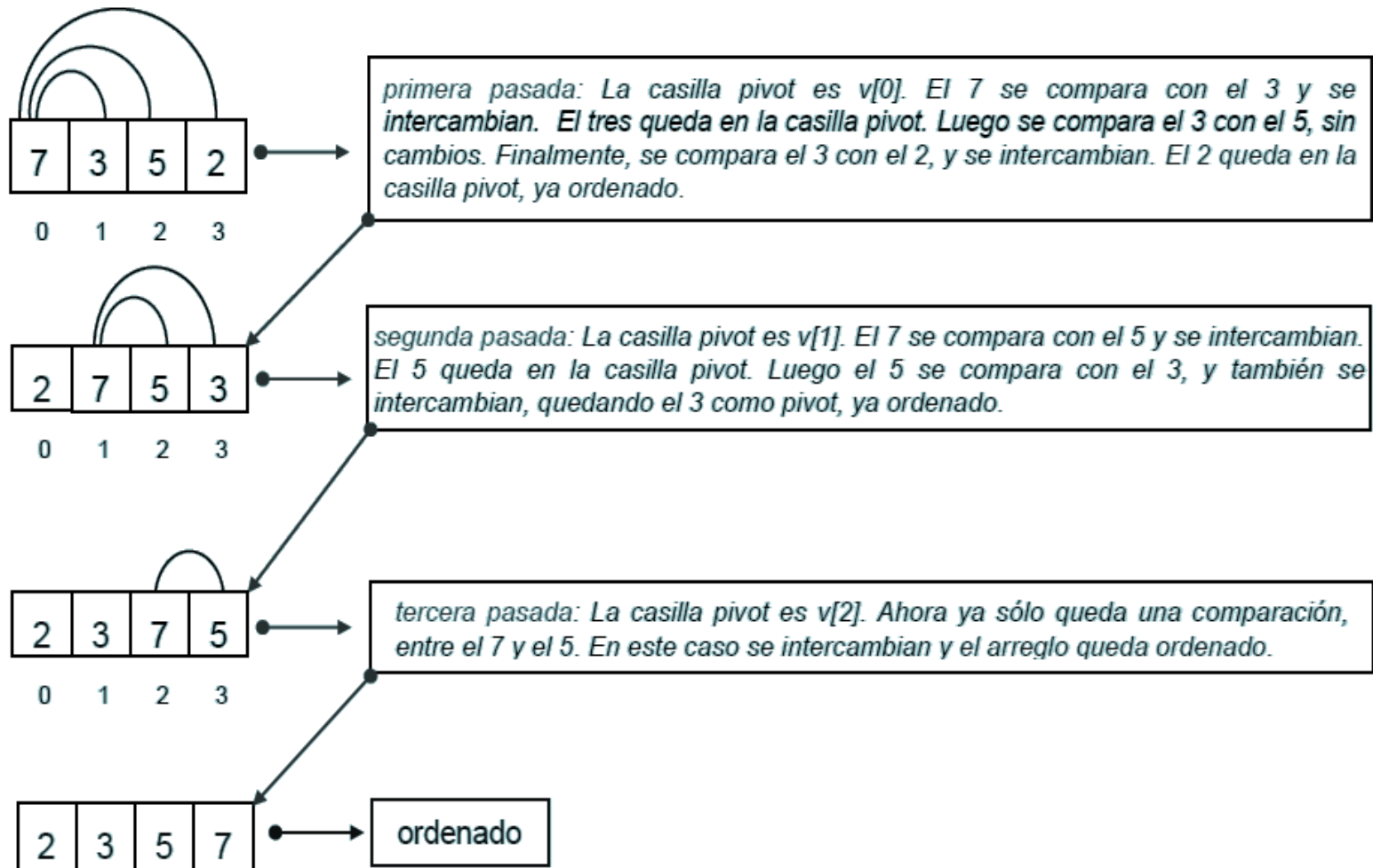
- Para visualizar los elementos almacenados en un arreglo se puede utilizar en forma directa la instrucción `print()`.
- Si la visualización de los elementos contenidos en un arreglo se quiere hacer de una manera mas personalizada, en estos casos se recurre a la utilización de un ciclo `for`. Ejemplo:

```
# Tomando como base el arreglo con 4 componentes anterior.  
for i in range(4):  
    # Visualización de elementos.  
    print('En la posición:', str(i), 'la nota es:', notas1[i])
```

Ordenamiento de arreglo unidimensional

- Una operación muy común cuando se trabaja con arreglos unidimensionales es el de **ordenar sus elementos** ya sea en orden ascendente o descendente.
- La ventaja del ordenamiento en cualquier estructura de datos radica fundamentalmente en la operación de búsqueda de un elemento. Ya que si la estructura tiene sus **elementos ordenados** la **búsqueda es más rápida**.
- Para realizar esta tarea existen diversos algoritmos, uno de ellos es el que se denomina **Ordenamiento de Selección Directa**.

Algoritmo de Ordenamiento: Selección Directa



Ordenamiento: Selección Directa. Implementación en Python

```
def selection_sort(v):  
    # ordenamiento por selección directa  
    n = len(v)  
    for i in range(n-1):  
        for j in range(i+1, n):  
            if v[i] > v[j]:  
                v[i], v[j] = v[j], v[i]
```

Ordenamiento: Selección Directa.

Ejemplificación

```
__author__ = 'Cátedra de AED'

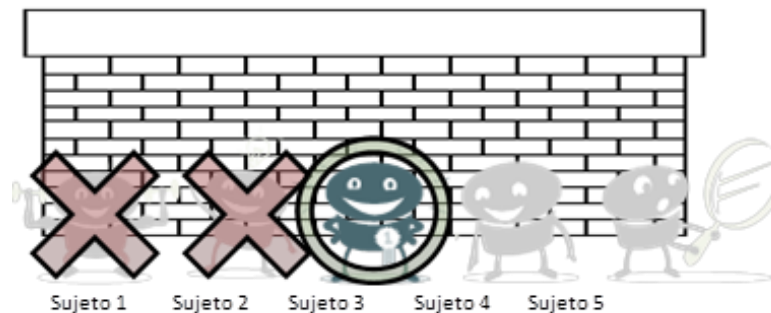
def principal():
    v = [3, 2, 7, 1]
    # Ordenar el arreglo.
    selection_sort(v)
    print('\nEl arreglo ordenado es:', v)

if __name__ == '__main__':
    principal()

# La visualización por consola es
[1,2,3,7]
```

Búsqueda Secuencial

- Este tipo de búsqueda permite resolver el problema de conocer si un determinado valor “x” se encuentra contenido o no un arreglo de “n” componentes.
- En esta búsqueda la mecánica a seguir es: si se encuentra el elemento se informa la posición y si no se encuentra se informa con un mensaje tal situación.
- Este tipo de búsqueda se puede aplicar cuando el arreglo esta desordenado o no se conoce nada respecto a su estado.



Búsqueda Secuencial.

Implementación en Python

```
# arreglos.py (módulo)

def linear_search(v, x):
    # busqueda secuencial...
    for i in range(len(v)):
        if x == v[i]:
            return i
    return -1
```

Algoritmo: Búsqueda Secuencial.

Ejemplificación

```
__author__ = 'Cátedra de AED'

import arreglos

def principal():
    v = [ 3, 2, 7, 1]
    x = int(input('Valor a buscar en el arreglo: '))
    # aplicar búsqueda secuencial...
    ind = arreglos.linear_search(v, x)
    # notificar el resultado de la búsqueda...
    if ind >= 0:
        print('\nEstá en la casilla', ind)
    else:
        print('\nNo está en el arreglo')

if __name__ == '__main__':
    principal()
```

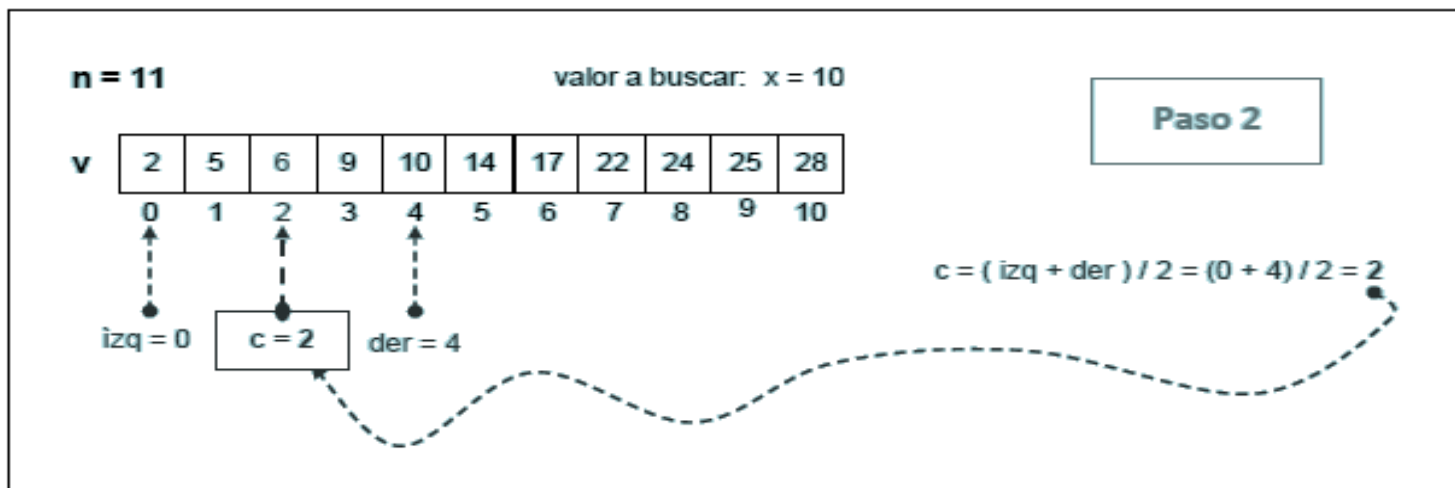
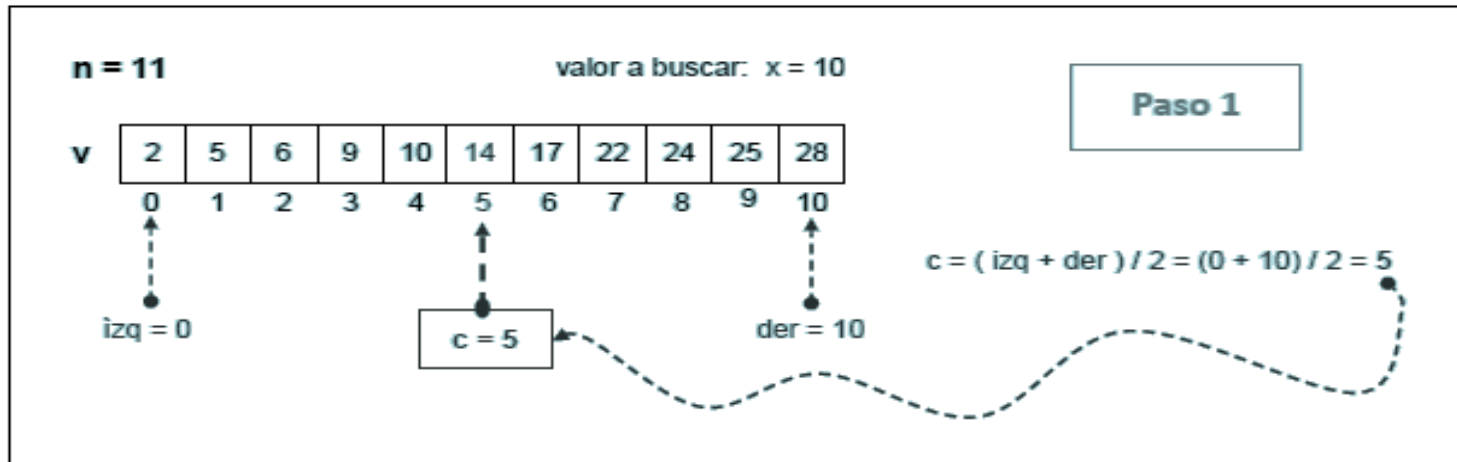
Búsqueda Binaria

- Este tipo de búsqueda es mucho mas eficiente que la búsqueda secuencial y solo puede ser aplicada si el arreglo “v” esta ordenado.
- La idea básica del algoritmo es:
 - Uso de dos índices auxiliares (izq y der).
 - En el intervalo marcado por izq y der se toma el elemento central cuyo índice es c:

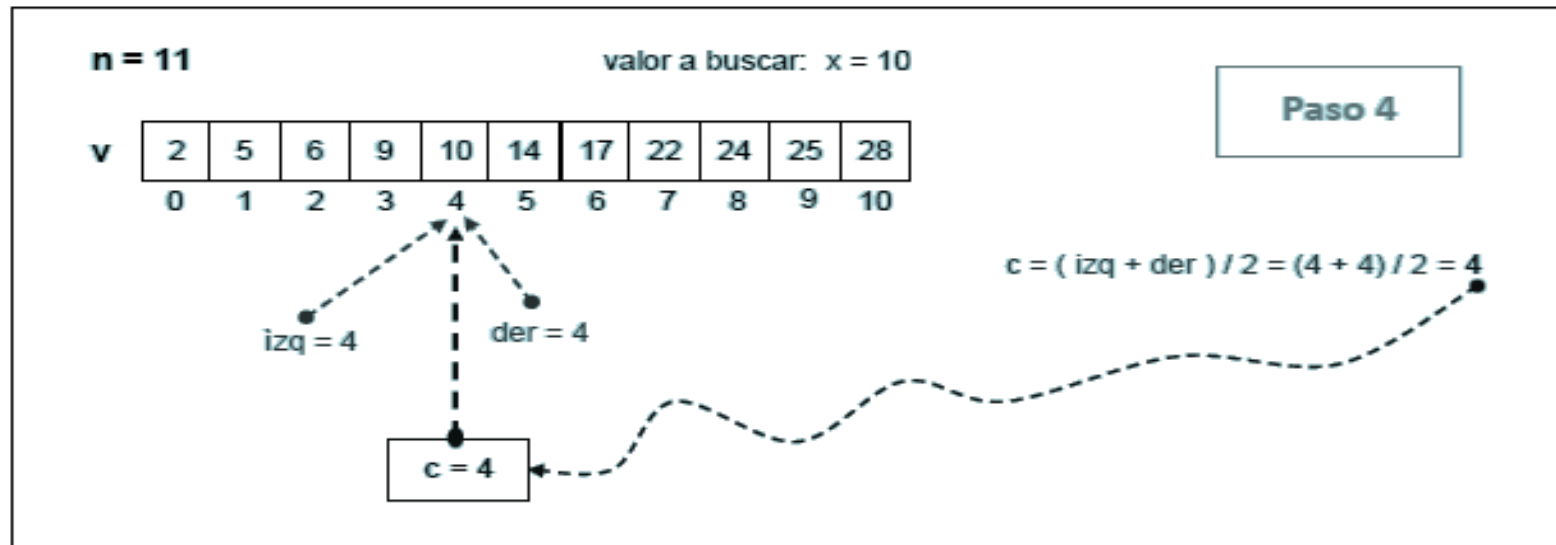
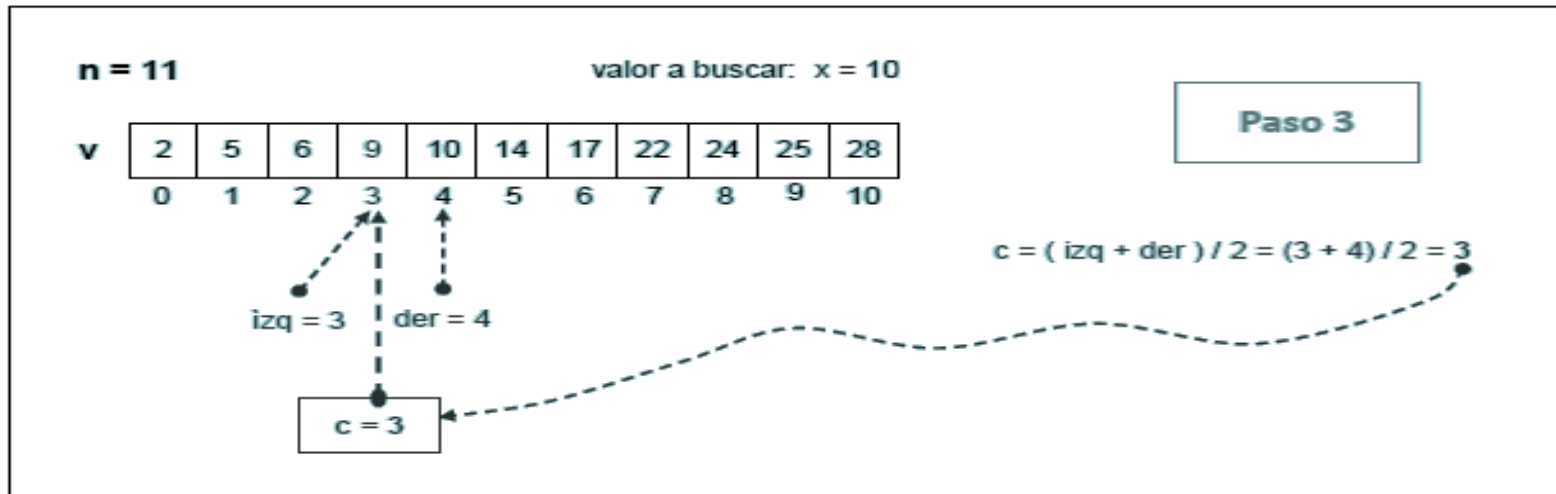
$$c = (izq + der) // 2$$

- Se verifica si el valor contenido en v [c] coincide con el número buscado. Si coincide termina la búsqueda. En caso contrario se aprovecha el ordenamiento del arreglo.

Esquema de trabajo: Búsqueda Binaria



Esquema de trabajo: Búsqueda Binaria



Búsqueda Binaria. Implementación en Python



```
# arreglos.py (módulo)

def binary_search(v, x):
    # busqueda binaria...
    izq, der = 0, len(v) - 1
    while izq <= der:
        c = (izq + der) // 2
        if x == v[c]:
            return c
        if x < v[c]:
            der = c - 1
        else:
            izq = c + 1
    return -1
```

Búsqueda Binaria. Ejemplificación

```
__author__ = 'Cátedra de AED'

import arreglos

def principal():
    v = [1, 2, 3, 7]
    x = int(input('Valor a buscar en el arreglo: '))
    # aplicar búsqueda binaria...
    ind = arreglos.binary_search(v, x)
    # notificar el resultado de la búsqueda...
    if ind >= 0:
        print('\nEstá en la casilla', ind)
    else:
        print('\nNo está en el arreglo')

if __name__ == '__main__':
    principal()
```

Muchas Gracias por la atención!!

Cualquier duda pueden contactarme por mensajería interna del Aula Virtual.